4. Functions, Multiple Returns, and Error Basics

04. Functions, Multiple Returns, and Error Basics

Phase: Phase 1 — CS fundamentals through Go
Estimated time: 1 day
Slug: `04-functions-errors`

Learning objectives

- Define functions with parameters and multiple return values
- Use the error interface as Go's primary failure signal
- Name return values when it improves clarity
- Avoid panic for ordinary errors

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Functions

```
func Add(a, b int) int {
    return a + b
}

func Div(a, b int) (int, error) {
    if b == 0 {
        return 0, errors.New("division by zero")
    }
    return a / b, nil
}
```

Idiom: last return value is often `error`. Callers check immediately:

```
v, err := Div(x, y)
if err != nil {
    return err
}
```

Why not exceptions?

Go's explicit errors make control flow visible. You see every failure path. `panic` exists for truly unrecoverable programmer mistakes — not for "file not found".

Named returns

```
func Split(sum int) (x, y int) {
    x = sum * 4 / 9
    y = sum - x
    return
}
```

Use sparingly; great for docs, easy to misuse with shadowing.

Common mistakes (especially from Python instincts)

- Ignoring `err` with _ habitually
- Using panic like Python raise for normal failures
- Returning error strings inconsistently (prefer `fmt.Errorf` / `errors.New`)

Exercises

Work in `exercises/04-functions-errors/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/04-functions-errors/` until you have an honest failing attempt.

Self-check

1. Why is `(T, error)` preferred over exceptions here?
2. What should you do immediately after calling a function that returns error?
3. When is panic appropriate?

Further reading

- https://go.dev/blog/error-handling-and-go
- https://go.dev/tour/basics